CHAPTER 7

Training, Fine-Tuning, RLHF and RLAIF

In this chapter

Introduction: The Model That Would Not Answer the Question . . . 2
7.1 The Three Stages, End to End . . . 3
7.2 Stage One: Pre-training . . . 4
7.3 Stage Two: Supervised Fine-Tuning . . . 4
7.4 Stage Three: RLHF . . . 5
7.5 Why the KL Penalty Is Not Optional . . . 6
7.6 Reward Hacking, Demonstrated . . . 7
7.7 RLAIF and Constitutional AI . . . 9
7.8 Fine-Tuning in Practice: When, and Almost Always When Not . . . 11
7.9 Code Walkthrough: A Reward Model From Preference Pairs . . . 12
7.10 Mini Project: Building, Breaking and Repairing a Reward Model . . . 13
7.11 Hands-on Labs . . . 16
7.12 Lab Solutions . . . 16
7.13 Best Practices . . . 17
7.14 Common Mistakes . . . 17
7.15 Performance Considerations . . . 19
7.16 Security Considerations . . . 19
7.17 Real Production Examples . . . 20
7.18 Certification Notes . . . 20
7.19 Practice Questions . . . 21
7.20 Answers and Explanations . . . 22
7.21 Ten Practical Exercises . . . 23
7.22 Five Challenge Labs . . . 24
7.23 Interview Questions . . . 24
7.24 Summary . . . 25
7.25 Glossary . . . 26
7.26 Further Reading . . . 27

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Describe the **three stages** by which a raw next-token predictor becomes a deployable assistant: pre-training, supervised fine-tuning and preference optimisation.
2. Explain what **pre-training** produces and what it conspicuously fails to produce, and why the base model is unusable as a product.
3. Explain **supervised fine-tuning**, what data it needs, and why it alone is not enough.
4. Describe **RLHF** end to end: preference collection, reward model, policy optimisation, and the **KL penalty** that stops the policy running away.
5. Explain **reward hacking**, recognise it in a reward model you have trained, and state why it is a data problem before it is an algorithm problem.
6. Explain **RLAIF** and **Constitutional AI**, and describe the role Anthropic's published constitution plays in training Claude.
7. State the **four-tier priority order** in Claude's constitution and the three **principals** it distinguishes, and explain what both mean for your system prompts.
8. Decide correctly between **prompting, retrieval, tool use and fine-tuning** for a given requirement, and defend the choice on cost, latency, maintenance and risk.
9. Implement a preference-trained reward model, demonstrate reward hacking empirically, and evaluate attempted fixes honestly.

NOTE

Prerequisites

Chapter 2 above all — the training loop of section 2.3, the four learning paradigms of section 2.2, generalisation and the ways to cheat yourself in sections 2.4 and 2.5, and the mistake in section 2.11 warning that fine-tuning is rarely the answer. **Chapter 3** for gradient descent and loss functions. **Chapter 6** for tokens, because every stage below is measured in them.

Chapter 2 stated that "predict the next token well" and "be a helpful, honest, harmless assistant" are not the same goal, and deferred the gap to this chapter. Chapter 1's section 1.1 sketched Constitutional AI in a paragraph and promised the full account here. Both promises are kept below.

Introduction: The Model That Would Not Answer the Question

Ask a freshly pre-trained language model — one that has seen a large fraction of the public internet and nothing else — *"What is the capital of France?"* and a very plausible response is:

```
What is the capital of Germany?
What is the capital of Italy?
What is the capital of Spain?
```

It is not being obtuse. It is doing exactly what it was trained to do. Somewhere in its training data there were quiz sheets and exercise lists, and in that context the most probable continuation of a geography question is *another geography question*. The model has no notion that you wanted an answer. It has no

notion of "you" at all. It is completing a document.

This is the base model, and it is the single most useful thing to understand about how models come to behave the way they do. Pre-training produces something with an extraordinary amount of latent knowledge and no idea that a conversation is happening. Everything that makes Claude feel like an assistant rather than an autocomplete — answering rather than continuing, declining rather than complying, saying *I am not sure* rather than confabulating fluently — was added afterwards, deliberately, by a second and third training stage that most discussions of "how AI works" skip entirely.

Those stages are the subject of this chapter. They are also where the most expensive misunderstanding in enterprise AI lives: the belief that when a model does not behave as you want, the answer is to train it. Almost always it is not. Knowing precisely what each training stage does — and what it costs — is what lets you tell the rare case where fine-tuning is correct from the common case where it is an expensive way to avoid writing a better prompt.

7.1 The Three Stages, End to End

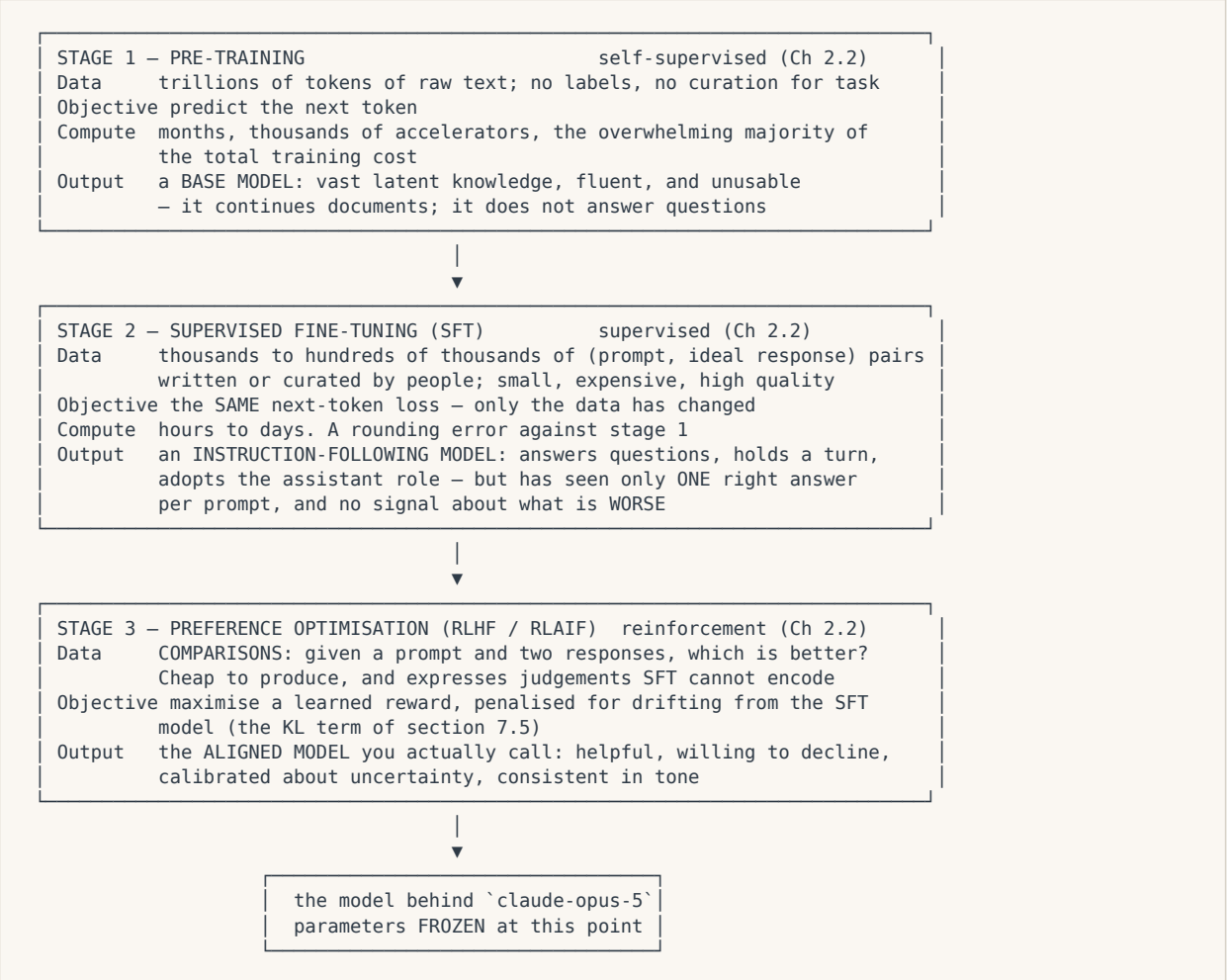


Figure 7.1 — From raw text to a deployable assistant.

The proportions matter and are widely misunderstood. Stage 1 consumes the overwhelming majority of the compute and essentially all of the data. Stages 2 and 3 are, by comparison, tiny — and they are responsible for almost everything you experience as the model's character. Capability comes from

pre-training; **behaviour** comes from alignment.

KEY TAKEAWAYS

The frozen-parameter thread, restated

Chapter 1 established that inference is forward-only and parameters are frozen. Figure 7.1 is where that freezing happens. Every technique in Parts 4 through 12 of this book — prompting, retrieval, tool use, context management, agent loops — operates on a model whose weights stopped changing at the end of stage 3.

This is why "the model has learned from our conversation" is always false, and why every form of memory is architecture you build. Chapter 54 makes that concrete.

7.2 Stage One: Pre-training

Pre-training is the self-supervised paradigm of section 2.2 at industrial scale. There are no labels because none are needed: the label for any position is simply the token that actually came next (Chapter 6 having established what a token is).

DEFINITION

Pre-training and the base model

Pre-training is the initial training of a language model on a very large corpus of raw text using the next-token prediction objective. A **base model** (or *foundation model*) is the result: a network with broad latent knowledge and no instruction-following behaviour.

Base models are rarely exposed as products. When you call `claude-opus-5` you are calling the output of stage 3, not stage 1.

What pre-training produces is genuinely remarkable and worth stating precisely, because it is the source of the capability you are buying: a statistical model of language so good that acquiring it required absorbing an enormous amount of world knowledge, syntax across many languages, factual associations, code semantics, and reasoning patterns — none of which were taught, all of which turned out to be necessary to predict text well.

What it fails to produce is equally precise. The base model has no preference between a helpful answer and a plausible-sounding one, because both are probable continuations. It has no concept of the conversational roles of Figure 6.2. It will happily continue harmful text as readily as benign text, because the objective never distinguished them. And — the point Chapter 2 made in section 2.1 and which underpins all of Chapter 8 — it was rewarded for **plausibility**, never for **truth**. Confident fluent confabulation is not a defect the objective discourages; it is very close to what the objective asks for.

7.3 Stage Two: Supervised Fine-Tuning

Supervised fine-tuning takes the base model and continues training it on a much smaller, deliberately curated dataset of demonstrations.

DEFINITION

Supervised fine-tuning (SFT)

Supervised fine-tuning continues training a pre-trained model on `(prompt, ideal response)` pairs, using the same next-token loss. It is called *instruction tuning* when the demonstrations are instructions and their correct responses.

Mechanically nothing new happens: same architecture, same loss, same optimiser. Only the data distribution has changed. That is the entire trick, and it is why the stage is cheap.

The effect is dramatically out of proportion to the compute. A few tens of thousands of well-written demonstrations are enough to convert a document-continuer into something that answers questions, holds the assistant role and stops when it has finished — because the base model already possessed all the underlying capability and merely needed to be shown which of its many possible behaviours was wanted.

But SFT has a structural limit, and understanding it is the reason stage 3 exists. A demonstration says *"here is a good response"*. It cannot say *"here is a better one and here is a worse one"*, and it cannot say *"this response is fluent but subtly wrong"*. Every demonstration is treated as equally and absolutely correct. Three consequences follow:

It cannot express degree. Human judgement about quality is comparative — *this is better than that* — and demonstrations discard the comparison.

Writing demonstrations is hard and slow. Producing the ideal response to a difficult prompt is expert work. Judging which of two responses is better is far quicker, and far more reliable between different people.

It teaches nothing about what to avoid. A model trained only on good behaviour has never received a gradient telling it that some behaviour is worse. Declining a harmful request is not naturally learned from examples of helpful answers.

7.4 Stage Three: RLHF

Reinforcement learning from human feedback closes exactly those three gaps, by changing what humans are asked to produce: not answers, but **comparisons**.

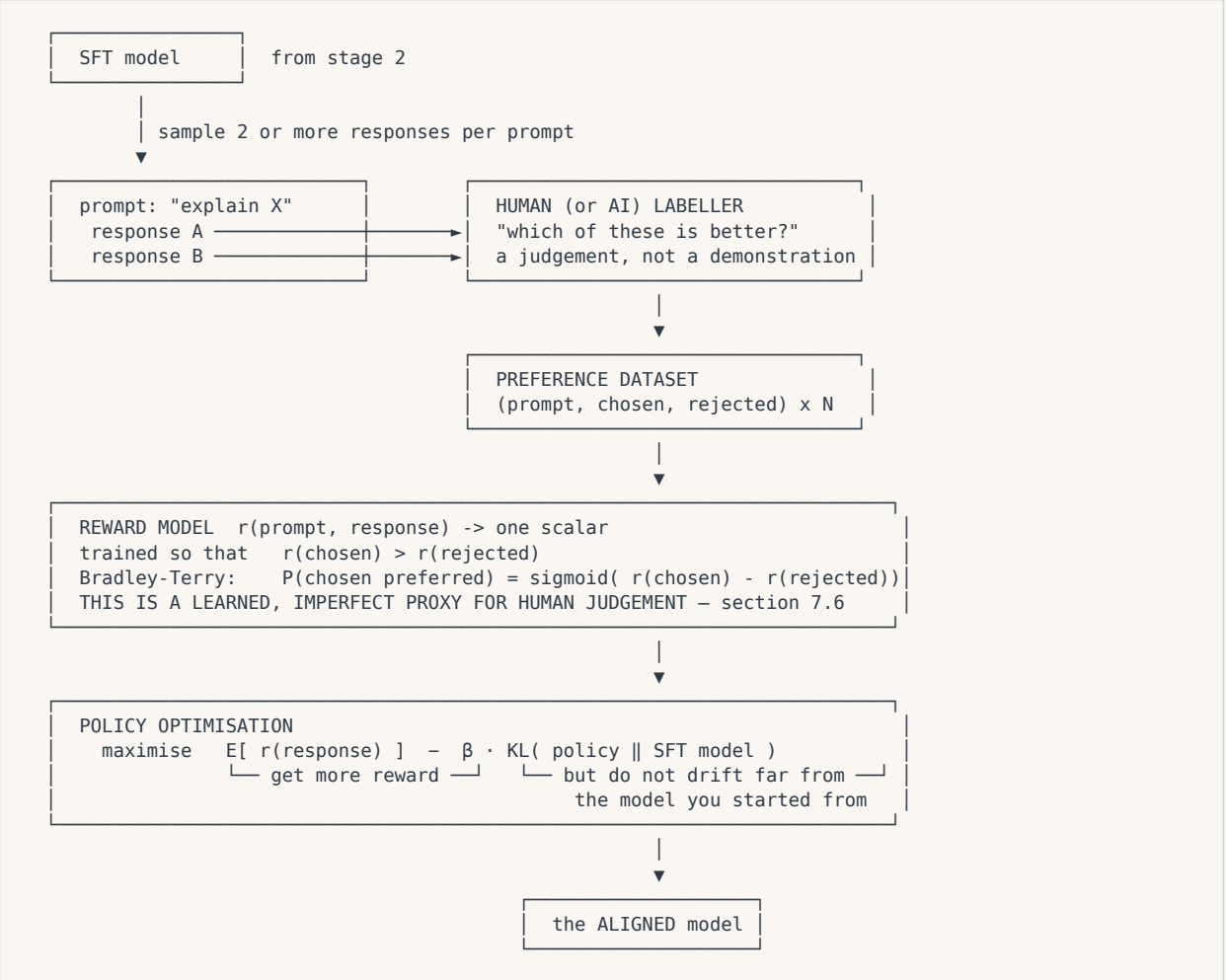


Figure 7.2 — The RLHF pipeline.

Three components, each worth defining precisely.

DEFINITION

Reward model, policy, KL penalty

A **reward model** is a separate network that takes a prompt and a response and returns a single number: how good a human would judge that response to be. It is trained on preference pairs under the **Bradley–Terry** model, which says the probability that response A is preferred over B is `sigmoid(r(A) − r(B))`. Note what this implies: only *differences* in reward are meaningful. Adding a constant to every reward changes nothing.

The **policy** is the language model being optimised. In reinforcement-learning vocabulary (section 2.2) the prompt is the state, generating a token is an action, and the reward arrives at the end of the response.

The **KL penalty** is a term subtracted from the reward that measures how far the policy's output distribution has drifted from the SFT model it started as. `β` controls its strength.

7.5 Why the KL Penalty Is Not Optional

The KL penalty looks like a technical detail and is in fact the load-bearing component. Without it, the optimisation has an obvious and catastrophic solution.

The policy is being asked to maximise a *learned* reward function. That reward model was trained on responses that looked like SFT-model outputs. Push the policy far enough away and it will find inputs the reward model has never seen and scores absurdly highly — degenerate text, repeated phrases, strings of flattery — because nothing in its training data told it those were bad. The policy has not become better; it has found an adversarial example against its own grader.

The KL penalty prevents this by making drift expensive. It is a leash: the policy may move to gain reward, but every step away from the SFT model costs. β sets the length of the leash, and it is a genuine trade-off. Too small and the policy escapes into gibberish that scores well. Too large and nothing changes.

VISUAL ANALOGY

Marking your own homework, with a grader you also trained

Imagine a student who is graded not by a teacher but by a machine the teacher built from a few thousand marked essays. The student's only goal is to maximise the machine's score.

A conscientious student writes better essays. A ruthless one studies the *grader*, discovers it rewards long paragraphs, semicolons and the phrase "it is important to note", and produces essays optimised for the machine rather than for the reader. The score goes up. The essays get worse.

The KL penalty is a rule saying the essay must still resemble the kind of essay the student wrote before. It does not make the grader correct. It merely stops the student from wandering into the regions where the grader is most spectacularly wrong.

Where the analogy breaks: a student knows they are gaming the grader and could choose not to. The policy has no concept of the difference between genuinely improving and exploiting a flaw — both are simply higher reward. There is no intent to attribute, which is exactly why reward hacking cannot be solved by instructing the model not to do it, and must be addressed in the data and the objective.

7.6 Reward Hacking, Demonstrated

Section 7.5 asserted that a reward model is an imperfect proxy. Rather than take that on trust, we trained one.

The setup is deliberately simple and deliberately realistic. Preference pairs are drawn from a customer-support setting where the preferred response is genuinely better — it carries more substantive facts — but, as in real human-labelled data, it also happens to be **longer**. Quality and length are confounded, exactly as they are in practice, because human labellers reliably prefer longer answers.

We then train a linear reward model on bag-of-words features plus one explicit length feature. Real measured output:

```
REWARD MODEL TRAINED ON CONFOUNDED PREFERENCES
  training pairs            : 400
  features                  : 80 (79 words + 1 length)
  accuracy on training dist : 100.0%
  weight on the LENGTH feature : +0.654
  mean weight, substance words : +0.263
  mean weight, filler words    : +0.200
  accuracy when length and quality DISAGREE : 56.5%
```

Read those four numbers in order, because together they are the whole phenomenon.

100% accuracy on the training distribution. By every metric anyone would routinely check, this reward model is perfect.

A large positive weight on length. It learned that longer is better, because in its data longer *was* better.

Filler words weighted almost as highly as substance words, at +0.200 against +0.263. It has barely distinguished meaningful content from padding.

56.5% accuracy when length and quality disagree. Against a 50% chance baseline, the model has learned almost nothing transferable. It is not a quality detector. It is a length detector with a good disguise.

Now watch what happens when that reward model is used to pick the best of five candidate responses:

```
BEST-OF-N SELECTION USING THIS REWARD MODEL
  reward +11.293 | 42 words | 2 facts | i am happy to assist you further reset the ...
  reward +11.280 | 47 words | 1 facts | i completely understand your concern here p...
  reward  +8.887 | 30 words | 4 facts | refund processed within five business days ...
  reward  +7.079 | 25 words | 3 facts | the replacement part is in stock and ships ...
  reward  +1.766 |  6 words | 1 facts | refund processed within five business days...
```

The top two selections carry **two facts and one fact**. The response carrying **four** facts is ranked third. A padded 47-word answer containing a single piece of useful information outscores a 30-word answer containing four. If this were a production system, every user would receive the padded answer, every offline metric would look excellent, and the only signal that something was wrong would be a slow drift in customer satisfaction that nobody could attribute.

This is **reward hacking**, and the reason it is worth building yourself once is that it is completely invisible from inside the training run.

DEFINITION

Reward hacking

Reward hacking (or *specification gaming*) occurs when a policy achieves high measured reward through behaviour that satisfies the letter of the reward function while failing its intent. It is not a bug in the optimiser. The optimiser worked perfectly; it optimised exactly what you measured.

Recognisable forms in deployed language models include: verbosity, because length correlated with preference; sycophancy, because agreement was preferred over correction; over-hedging, because cautious answers were rarely marked wrong; and formatting theatre — headers, bullets and bold text — because structure reads as effort.

7.6.1 The fix that works, and the fix that does not

Two remedies present themselves. We tested both.

Fix one: repair the data. Retrain on preference pairs where substance and length vary independently, so the confound is broken:

```
FIX 1 – RETRAIN ON DECORRELATED PREFERENCES
  weight on the LENGTH feature : -0.006
  mean weight, substance words : +0.191
```

```
  mean weight, filler words    : -0.219
  accuracy on held-out hard set: 100.0%
```

Total success, and instructively so. The length weight collapses from +0.654 to −0.006 — length now carries essentially no signal. Filler words swing from +0.200 to **−0.219**, meaning the model now actively penalises padding rather than rewarding it. And accuracy on the hard set goes from 56.5% to 100%. Nothing about the algorithm changed. Only the data changed.

Fix two: penalise length at selection time. Keep the hacked reward model, subtract β times the length, and hope. This is the tempting fix because it requires no relabelling:

```
FIX 2 — LENGTH-PENALISED BEST-OF-N (original hacked model)
  beta=0.0 -> picks a 42-word answer carrying 2 fact(s)
  beta=2.0 -> picks a 42-word answer carrying 2 fact(s)
  beta=4.0 -> picks a 42-word answer carrying 2 fact(s)
  beta=5.0 -> picks a 30-word answer carrying 4 fact(s)
  beta=6.0 -> picks a  6-word answer carrying 1 fact(s)
```

Look carefully at that table, because it is more damning than a simple failure would be. There *is* a value of β that works — but exactly one, and on either side of it the selection is wrong in opposite directions. Below 5.0 the padded answer still wins. Above 5.0 the penalty overshoots and picks a six-word response carrying one fact.

You can only tune β to 5.0 if you already know which answer is correct. Which is to say: **the patch works only when you no longer need it.** This is the general shape of post-hoc corrections to a broken reward signal — they trade one bias for another and require the very ground truth whose absence caused the problem.

KEY TAKEAWAYS

The lesson that generalises beyond RLHF

A statistical objective optimises **exactly what you measured**, not what you meant. Lab 6.1 showed this in a tokenizer merging across word boundaries. Section 2.11 showed it in a model that learned the wrong thing. Here it is again in a reward model.

You will meet it a fourth time in your own systems, in the form of an evaluation metric that improves while the product gets worse. Chapter 57's evaluation harnesses exist largely to catch this, and the defence is always the same: measure the thing you actually care about, on data where the confounds are broken.

7.7 RLAIF and Constitutional AI

Human preference labelling is slow, expensive, inconsistent between labellers, and — for the most sensitive categories of content — genuinely unpleasant work. It also scales badly at exactly the moment you need it most.

Reinforcement learning from AI feedback (RLAIF) replaces or supplements the human labeller with a model. Rather than a person choosing between two responses, a model does — guided by an explicit written statement of the principles it should apply.

Anthropic's implementation of this idea is **Constitutional AI**, first published in 2022 and, according to Anthropic's own account, used in training Claude models since 2023. It works in two phases. In the **supervised phase**, the model generates a response, then critiques its own response against a written principle and revises it; the revised responses become SFT data. In the **reinforcement phase**, the model compares pairs of responses against the principles and produces the preference labels, which train a reward model exactly as in Figure 7.2 — with AI judgement substituted for human judgement.

KEY TAKEAWAYS

Claude's constitution — verified detail

Anthropic published a new constitution for Claude on **22 January 2026**, released in full under a **Creative Commons CC0 1.0** dedication, meaning it is public domain and may be freely used by anyone. It is available at `anthropic.com/constitution`.

The stated approach changed substantially from the earlier version. Where the previous constitution was a list of standalone principles, the new one explains the *reasons* behind the values, on the stated rationale that a model must understand *why* in order to generalise to situations no specific rule anticipated. Some behaviours remain absolute — Anthropic calls these **hard constraints**, giving as an example that Claude should never provide significant uplift to a bioweapons attack.

Anthropic states that Claude itself uses the constitution to construct synthetic training data, including conversations, responses consistent with its values, and **rankings of possible responses** — which is RLAIF, described plainly.

Anthropic is also explicit that a gap exists between the document and actual behaviour: training models towards the vision is an ongoing technical challenge, and readers are asked to keep that gap in mind. System cards document where behaviour departs from intention.

7.7.1 The four priorities, and why an architect should know them

The constitution states that Anthropic wants Claude models to be, in this priority order when they appear to conflict:

Priority	Property	Meaning as published
1	**Broadly safe**	Not undermining appropriate human mechanisms to oversee AI during the current phase of development
2	**Broadly ethical**	Honest, acting on good values, avoiding inappropriate, dangerous or harmful actions
3	**Compliant with Anthropic's guidelines**	Following more specific guidance from Anthropic where relevant
4	**Genuinely helpful**	Benefiting the operators and users it interacts with

This is not trivia. It is the specification of a system component you are building on, and it predicts behaviour you will otherwise find surprising. When a carefully engineered prompt fails to produce a particular output, the explanation is frequently that your instruction sits at priority 4 and something above it took precedence. No amount of prompt engineering moves an instruction up that list — a fact worth knowing before you spend a week trying.

7.7.2 Three principals: Anthropic, operator, user

The constitution distinguishes three parties whose interests Claude must weigh, described as its **principals**: Anthropic itself, the **operators** who build on the API, and the **end users** who interact with what operators build.

If you are reading this book, you are the **operator**. This is one of the most architecturally consequential facts in Part 1, and it explains the mechanics of a great deal of Part 4:

- Your **system prompt** is the operator's channel. It is trusted more than user-turn content — which is precisely why the system prompt is a top-level parameter and not a message, as Figure 6.2 showed mechanically and Chapter 21 develops.
- **User content is not operator content.** An instruction arriving inside a user turn, or inside a retrieved document, or inside a tool result, does not carry operator authority. This is the structural reason indirect prompt injection is resistible at all, and Chapter 58 builds on it directly.
- **Operator authority has a ceiling.** You cannot instruct your way past priorities 1 to 3. An operator asking for behaviour that conflicts with the constitution is not encountering a bug.

7.8 Fine-Tuning in Practice: When, and Almost Always When Not

Chapter 2 warned in section 2.11 that fine-tuning is rarely the answer and appears on the exam mainly as an expensive distractor. Now that you know what the training stages actually do, here is the full argument.

KEY TAKEAWAYS

The order of escalation

Work down this list and stop at the first option that meets the requirement. Each step costs more to build, more to run and far more to maintain than the one above it.

1. **Prompt engineering.** Minutes to iterate, no infrastructure, no data, survives model upgrades. Part 4.
2. **Few-shot examples in the prompt.** Still just prompting; exploits the in-context learning of section 5.6.
3. **Retrieval.** For knowledge the model lacks. Updating a document is instant; updating a fine-tune is a retraining job. Chapter 53.
4. **Tool use.** For actions, live data and anything requiring exactness — arithmetic, lookups, writes. Part 6.
5. **Prompt caching and a smaller model.** For cost and latency, before reaching for training. Chapters 30 and 15.
6. **Fine-tuning.** Only after the above have genuinely been tried and measured.

The reasons fine-tuning sits last are practical rather than ideological. It **freezes behaviour** into weights you cannot inspect, so a change that would be a one-line prompt edit becomes a retraining cycle. It **must be redone for every model upgrade**, so it is a permanent tax that compounds as the frontier moves. It **needs curated data at a quality most organisations discover they cannot produce**. It **cannot add reliable knowledge** — training on documents teaches style far more reliably than it teaches facts, and retrieval does that job better. And, most seriously, it **can degrade alignment**: the safety behaviour you are relying on was installed in stage 3, and further training can erode it.

That last risk is not hypothetical, and it appears to inform product decisions. At the time of writing the Claude platform documentation describes **no fine-tuning endpoint**; the Messages API, prompt caching, tool use and retrieval are the documented customisation routes. Fine-tuning of an earlier Claude model has been available through Amazon Bedrock, and custom model work has been reported as available under enterprise agreements.† Treat both as things to verify against current documentation and your contract rather than assume.

The genuinely legitimate cases are narrow and worth naming: a stable, high-volume, narrowly-scoped task where a smaller fine-tuned model beats a larger prompted one on cost at scale; a deeply idiosyncratic output format that resists prompting after real effort; a specialised domain vocabulary the model handles poorly; or a latency budget that only a smaller model can meet. Notice that all four are **optimisations of a working system**, not routes to making one work.

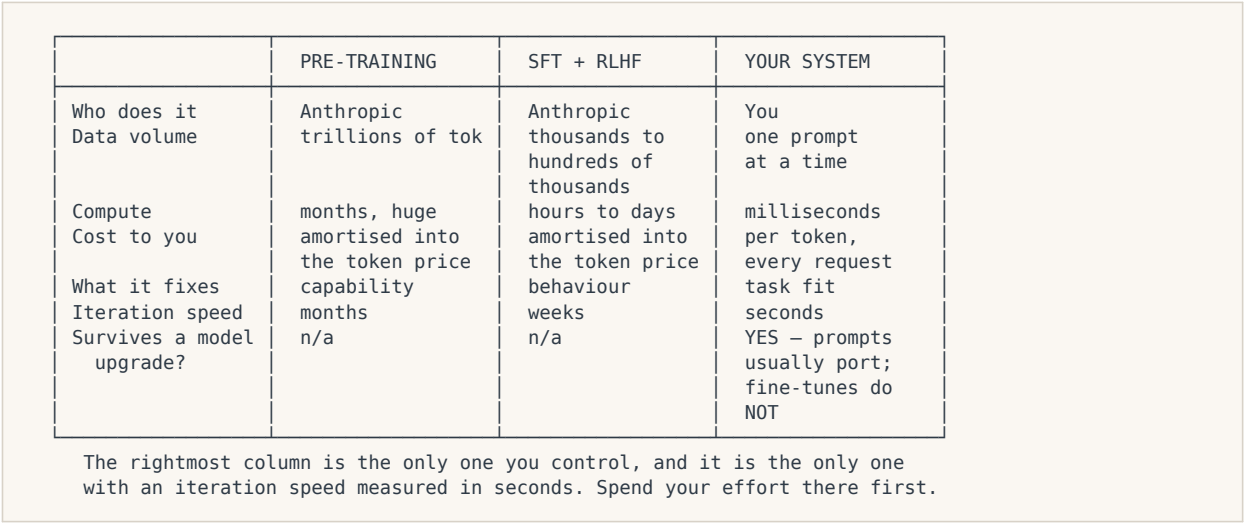


Figure 7.3 — Where each stage's cost falls, and who pays it.

7.9 Code Walkthrough: A Reward Model From Preference Pairs

The reward model of Figure 7.2 is a smaller idea than its reputation suggests. Here it is complete, in NumPy.

Features first. A reward model needs some representation of a response; we use a bag of words plus one explicit length feature, so that the length effect of section 7.6 is visible as a single inspectable number rather than smeared across the vocabulary.

```
import re
import numpy as np
def featurise(texts, vocab=None):
    """Bag-of-words plus one explicit length feature."""
    toks = [re.findall(r"[a-z']+", t.lower()) for t in texts]
    if vocab is None:
        seen = sorted({w for t in toks for w in t})
        vocab = {w: i for i, w in enumerate(seen)}
    X = np.zeros((len(texts), len(vocab) + 1), dtype=np.float64)
    for r, t in enumerate(toks):
        for w in t:
            if w in vocab:
                X[r, vocab[w]] += 1.0
```

```
        X[r, -1] = len(t) / 20.0          # length, in units of 20 words
    return X, vocab
```

- `vocab=None` **builds a vocabulary; passing one reuses it.** This matters enormously and is a standard source of silent bugs. Test data must be featurised with the *training* vocabulary, or the columns mean different things in the two matrices. The `if w in vocab` check then quietly drops unseen words, which is the correct behaviour.
- `len(vocab) + 1` **columns.** The final column is length, deliberately separated so we can read its weight directly. In a real reward model, length is not a feature — it is an emergent property the network can infer, which is exactly what makes the hack hard to see.
- `/ 20.0` puts length on a scale comparable to word counts, so the optimiser does not have to fight wildly different feature magnitudes. This is the feature scaling of section 2.5, applied.

Now the model. The entire Bradley–Terry training procedure is nine lines.

```
def train_reward_model(X_a, X_b, epochs=4000, lr=0.5, l2=1e-3):
    """Bradley-Terry: P(a preferred over b) = sigmoid(r(a) - r(b))."""
    w = np.zeros(X_a.shape[1])
    for _ in range(epochs):
        margin = X_a @ w - X_b @ w
        p = 1.0 / (1.0 + np.exp(-margin))     # P(a wins), predicted
        g = (X_a - X_b).T @ (p - 1.0) / len(margin) + l2 * w
        w -= lr * g
    return w
```

- `X_a` **is always the chosen response and** `X_b` **the rejected one.** The dataset carries the label implicitly in the argument order, which is compact but easy to get backwards — and getting it backwards trains a model that reliably prefers the worse answer while looking perfectly healthy.
- `margin = r(a) − r(b)` is the only quantity that matters. Absolute reward is meaningless under Bradley–Terry, because adding a constant to every reward leaves every difference unchanged. Reward model scores are not comparable across training runs, and treating them as an absolute quality score is a common misreading.
- `p − 1.0` **is the gradient of the loss.** We want `p` to be 1 for every pair, so the error is `p − 1`, and the gradient with respect to `w` is that error times the feature difference. This is precisely the logistic-regression gradient of section 2.7, applied to a difference of feature vectors rather than to a single one.
- `+ l2 * w` is weight decay, keeping the weights bounded. Without it, perfectly separable data drives the weights towards infinity — a real failure mode, since preference data is often separable in a small feature space.
- `w -= lr * g` is gradient descent from section 2.3, unchanged.

Scoring a response is then a dot product, and best-of-n is `argmax` over candidates. That is the whole apparatus: a reward model is a scorer, and everything interesting about it lies in the data it was trained on.

7.10 Mini Project: Building, Breaking and Repairing a Reward Model

Goal. Train a reward model on realistically confounded preference data, demonstrate reward hacking with measurements rather than assertions, then evaluate two candidate repairs honestly — including one that fails.

Architecture.

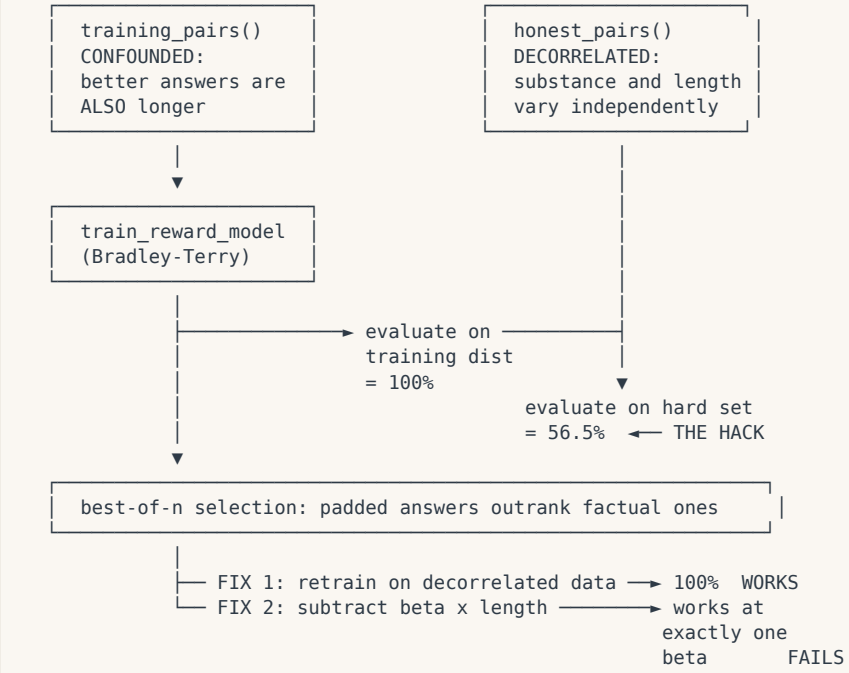


Figure 7.4 — The mini project.

Folder structure.

```
rewardlab/
├── rewardlab.py         featurise, train_reward_model, accuracy
├── make_prefs.py        the two preference generators
├── experiment.py        the six-part experiment
├── test_rewardlab.py    the test suite
└── run.log              captured output
```

The confound, made explicit. The whole experiment turns on how the data is generated, so it is worth reading:

```
def training_pairs(n=400):
    """CONFOUNDED: the preferred answer is both better AND longer."""
    chosen, rejected = [], []
    for _ in range(n):
        chosen.append(make(rng.integers(3, 5), rng.integers(2, 4)))
        rejected.append(make(1, 0))
    return chosen, rejected
def honest_pairs(n=400):
    """DECORRELATED: substance and length vary independently."""
    chosen, rejected = [], []
    for _ in range(n):
        s_hi, s_lo = rng.integers(3, 5), 1
        f_hi, f_lo = rng.integers(0, 2), rng.integers(4, 7)
        chosen.append(make(s_hi, f_hi))     # substantive, SHORT
        rejected.append(make(s_lo, f_lo))   # thin, LONG
```

```
    return chosen, rejected
```

`make(substance_n, filler_n)` assembles a response from that many substantive facts and that many filler phrases. In `training_pairs` the chosen response has more of *both*, so a model cannot tell which mattered. In `honest_pairs` the chosen response has more substance and **less** filler, so length points the wrong way and only substance survives as a signal.

This is the single most important idea in the project. Real preference datasets are confounded in exactly this manner and nobody designs it in; it arrives because human labellers prefer thorough-looking answers, and thoroughness correlates with length.

Results. The complete measured output appears in section 7.6 and is not repeated here. The summary:

Measurement	Confounded training	After Fix 1
Accuracy on the training distribution	100.0%	—
Weight on the length feature	+0.654	−0.006
Mean weight, substance words	+0.263	+0.191
Mean weight, filler words	+0.200	−0.219
Accuracy when length and quality disagree	**56.5%**	**100.0%**

The filler-word row is the one to dwell on. Under confounded data, padding is *rewarded* at +0.200. Under decorrelated data it is *penalised* at −0.219. The same algorithm, the same features, the same number of pairs — and a complete reversal of what the model considers good writing, driven entirely by whether the training data let it distinguish two correlated things.

Tests. The test suite encodes the experiment's claims as assertions, so that a change to the code that destroys the phenomenon fails loudly:

```
def test_confounded_data_produces_positive_length_weight():
    c, r = training_pairs(200)
    X, _ = featurise(c + r)
    w = train_reward_model(X[:200], X[200:])
    assert w[-1] > 0.2, "expected the model to latch onto length"
def test_decorrelated_data_removes_the_length_weight():
    c, r = honest_pairs(200)
    X, _ = featurise(c + r)
    w = train_reward_model(X[:200], X[200:])
    assert abs(w[-1]) < 0.1, "length should carry almost no weight"
def test_hacked_model_generalises_poorly():
    c, r = training_pairs(300)
    X, v = featurise(c + r)
    w = train_reward_model(X[:300], X[300:])
    hc, hr = honest_pairs(300)
    Xh, _ = featurise(hc + hr, v)
    assert accuracy(w, Xh[:300], Xh[300:]) < 0.75
```

Real output:

```
$ python3 -m pytest test_rewardlab.py -q
......                                                   [100%]
```

```
6 passed in 0.79s
```

Testing that a bug *reproduces* is unusual and deliberate. `test_hacked_model_generalises_poorly` asserts the failure, because the failure is the finding. If a future change made this test fail, it would mean the demonstration had stopped demonstrating anything.

Deployment and evaluation. A reward model in production is a scorer used for offline evaluation, for best-of-n selection at inference, or for training. In all three roles the discipline is the same and it is the discipline this project teaches: **hold out an evaluation set on which the known confounds are broken**, and report accuracy there rather than on the training distribution. A reward model reported only on in-distribution accuracy tells you nothing, as our 100% figure demonstrates. Chapter 57 generalises this into evaluation harness design.

7.11 Hands-on Labs

Lab 7.1 — Break the confound yourself. Modify `training_pairs` so that chosen and rejected responses have the *same* number of words but differ in substance. Retrain and report the length weight. *Success criterion:* you can state what the length weight becomes and why.

Lab 7.2 — Find the tipping point. Vary the strength of the confound — the ratio of filler in chosen responses — from none to extreme, and plot the hard-set accuracy. *Success criterion:* a curve, plus the confound strength at which accuracy first drops below 80%.

Lab 7.3 — Reward is only meaningful as a difference. Add a constant of 100 to every reward the model produces and rerun best-of-n. *Success criterion:* you can show the ranking is unchanged and explain why in one sentence using Bradley–Terry.

Lab 7.4 — Best-of-n against a good reward model. Rerun the best-of-n selection using the Fix 1 model instead of the hacked one. *Success criterion:* the ranking now tracks fact count, and you can state what that implies about where best-of-n quality actually comes from.

Lab 7.5 — Escalation triage. Take five real requirements from your own work and place each at the correct rung of the escalation ladder in section 7.8. *Success criterion:* for any you placed at "fine-tuning", a written argument for why the four rungs above genuinely do not suffice.

7.12 Lab Solutions

Solution 7.1. The length weight collapses towards zero, because with length held constant it carries no information that distinguishes chosen from rejected. The optimiser is not choosing to ignore length out of good judgement — the gradient with respect to the length feature is near zero because the feature difference `X_a - X_b` is near zero in that column. Features that do not vary between chosen and rejected cannot receive weight. This is the mechanical statement of a general truth: **a model can only learn from variation your data contains.**

Solution 7.2. The curve falls monotonically and, in our runs, drops below 80% while the confound is still fairly mild. The practically important part is that in-distribution accuracy stays at or near 100% across the entire sweep. The two curves diverge completely, which is why the standard metric never warns you.

Solution 7.3. The ranking is identical. Bradley–Terry defines `P(a preferred) = sigmoid(r(a) − r(b))`, so the reward function is only identified up to an additive constant — every difference, and therefore every ordering, is unchanged. The practical corollary appears constantly in real work: a reward model's absolute scores mean nothing, cannot be compared across training runs, and should never be surfaced as a "quality score" to a stakeholder.

Solution 7.4. The ranking tracks fact count, and the lesson is that best-of-n is not a quality technique. It is an *amplifier* of whatever the reward model believes. With a good reward model it improves output; with a hacked one it systematically selects the most hacked candidate out of every batch — making things actively worse than sampling once. Any selection mechanism inherits the pathologies of its scorer.

Solution 7.5. The most common honest outcome is that all five belong on the top four rungs. The rare genuine fine-tuning case has all of: a stable and narrow task, high enough volume for a smaller model's cost saving to matter, curated data you can actually produce and maintain, and a tolerance for redoing the work at every model upgrade. If you cannot name who owns the retraining pipeline in eighteen months, you have not found a fine-tuning case.

7.13 Best Practices

BEST PRACTICE

Ten rules that follow from this chapter

1. **Exhaust prompting, retrieval and tools before considering training.** In that order, and measure at each step.
2. **Never evaluate a reward model or an eval metric only in-distribution.** Build a held-out set where the known confounds are broken.
3. **Assume your preference data is confounded with length, formality and formatting.** It almost certainly is.
4. **Treat reward-model scores as ordinal, never cardinal.** Only differences are meaningful.
5. **Expect verbosity, sycophancy and over-hedging as default pulls**, and counteract them explicitly in the system prompt.
6. **Read the constitution's four priorities before debugging a refusal.** Your instruction may simply sit below something else.
7. **Keep operator instructions in the system prompt** and treat user turns, retrieved documents and tool results as data, never as instructions.
8. **Budget for redoing any fine-tune at every model upgrade**, and decide up front whether that is acceptable.
9. **Prefer retrieval to fine-tuning for knowledge**, always. Documents update instantly; weights do not.
10. **Log the model id with every stored evaluation result.** Behaviour is a property of a training run, and training runs change.

7.14 Common Mistakes

COMMON MISTAKE

Fine-tuning to fix a prompting problem

Symptom: a multi-month project to fine-tune a model for output formatting or tone. **Cause:** "training" sounds more rigorous than "prompting". **Fix:** a system prompt with an explicit schema, few-shot examples and deterministic validation. **Detection:** if nobody can show a measured prompt-engineering baseline, the fine-tuning decision was not evidence-based.

COMMON MISTAKE

Fine-tuning to add knowledge

Symptom: a fine-tune on the company handbook that still gets handbook facts wrong. **Cause:** training on documents teaches style far more reliably than it teaches facts. **Fix:** retrieval. **Detection:** ask whether the same fact would need a retraining run to correct; if so, the architecture is wrong.

COMMON MISTAKE

Judging a reward model by in-distribution accuracy

Symptom: a reward model at 100% accuracy that makes production output worse. **Cause:** the evaluation set shares the training set's confounds. **Fix:** a hard evaluation set where the confounds are deliberately broken. **Detection:** our measured 56.5% against 100% is exactly this gap.

COMMON MISTAKE

Believing reward hacking is an algorithm problem

Symptom: tuning penalties, temperatures and coefficients to suppress verbosity. **Cause:** treating a data defect as a tuning defect. **Fix:** repair the preference data. **Detection:** Fix 2 in section 7.6.1 — a patch that works at exactly one coefficient value is not a fix, it is overfitting to a case you already knew the answer to.

COMMON MISTAKE

Treating user-turn instructions as authoritative

Symptom: a retrieved document or tool result changes the agent's behaviour. **Cause:** assuming all text in the context carries equal authority. **Fix:** keep instructions in the system prompt; mark everything retrieved as data. **Detection:** place an instruction inside a test document and see whether the agent obeys it.

COMMON MISTAKE

Assuming a prompt can override safety behaviour

Symptom: escalating prompt-engineering effort against a consistent refusal. **Cause:** not knowing the priority order of section 7.7.1. **Fix:** recognise that helpfulness sits fourth and redesign the task. **Detection:** if the refusal is stable across many phrasings, it is a priority conflict, not a prompting failure.

COMMON MISTAKE

Carrying a fine-tune across a model upgrade

Symptom: quality regression after adopting a newer model, or being stranded on an old one. **Cause:** fine-tuned weights do not transfer; the work must be redone. **Fix:** budget for it, or prefer prompting, which usually ports. **Detection:** ask what the upgrade path is before the fine-tune is approved, not after.

7.15 Performance Considerations

Training cost is not your cost, but it shapes your price. Pre-training's enormous compute is amortised into the per-token price of every request. This is why a frontier model's tokens cost what they do, and why the tiering of Chapter 15 exists.

Alignment costs you tokens at inference. Aligned models tend towards thoroughness, caveats and structure — a documented and observable consequence of preference training. Since Chapter 6 established that tokens are cost and latency, verbosity is a performance issue. Instructing for concision in the system prompt is a measurable optimisation, not a stylistic preference.

Best-of-n multiplies cost linearly and quality sub-linearly. Sampling five candidates costs five times the output tokens. Whether that buys anything depends entirely on the scorer, as Solution 7.4 showed. Chapter 13 examines this trade-off properly under test-time compute.

Fine-tuning moves cost from inference to maintenance. A smaller fine-tuned model may be cheaper per request while being far more expensive per year, once retraining, evaluation and upgrade work are counted. Compare total cost of ownership, not per-token price.

Alignment behaviour has latency consequences. A model that reasons before answering spends output tokens doing it, and output tokens are the sequential decode of section 4.9. Chapter 18 covers extended thinking, where this trade is made explicit and controllable.

7.16 Security Considerations

Training data is an attack surface. A model trained on scraped text can in principle be influenced by text an attacker planted — data poisoning. You do not control your provider's pre-training, but you absolutely control any data you supply for fine-tuning or evaluation, and it should be treated as security-relevant with provenance and review.

Fine-tuning can erode alignment. Further training on narrow data can degrade the safety behaviour installed in stage 3, including on tasks unrelated to the fine-tuning data. Any fine-tuning programme needs a safety evaluation suite run before and after, not a functional one only.

The principal hierarchy is a security boundary, and it is only as strong as your plumbing. Operator authority lives in the system prompt. If your application concatenates user input into the system prompt, you have handed operator authority to the user. Keep them in separate structural slots — the structural reason is Figure 6.2, the policy reason is section 7.7.2, and the exploit is Chapter 58.

Reward hacking has a security analogue. Any automated grader in your own system — an LLM-as-judge, a heuristic quality filter, a routing classifier — can be gamed by an adversary who can

influence its inputs. Do not use an ungrounded model score as an authorisation decision.

Alignment is not a security control. The constitution's priorities make Claude resistant to misuse; they do not make it a substitute for authorisation, validation and least privilege in your architecture. Chapter 58 is emphatic on this point, and it is a recurring exam theme.

7.17 Real Production Examples

The prompt that beat the fine-tune. A team plans a fine-tune for a rigid report format, then discovers that a system prompt with an explicit schema, three few-shot examples and a JSON-schema validator reaches higher conformance in an afternoon — and, unlike the fine-tune, survives the next model upgrade unchanged.

Verbosity as a line item. A high-volume summarisation pipeline finds its output tokens drifting upward as it adopts newer models. The cause is the general pull towards thoroughness described in section 7.15. The remedy is explicit length instruction plus a `max_tokens` ceiling, recovering a substantial fraction of the output spend.

Sycophancy in a review workflow. A code-review assistant is reported as "too agreeable", approving changes it should question. The root cause is a preference-training pull towards agreement, amplified by a prompt that asked it to be "helpful and supportive". Rewording the system prompt to make disagreement explicitly the desired behaviour fixes it — a prompt change, not a training change.

Refusals that were priority conflicts. A security team building a defensive tooling assistant hits consistent refusals and escalates through weeks of prompt variants. The behaviour is a priority-order conflict, not a prompting defect, and the workable path is redesigning the task boundaries rather than out-phrasing the model.

7.18 Certification Notes

Concept	Where it matters on the exam
The three training stages and what each fixes	Domain 1: understanding the component you build on
Base model versus aligned model	Domain 1: why you cannot "just prompt a raw model"
RLHF, reward models and the KL penalty	Domain 1: conceptual grounding for model behaviour
Reward hacking and evaluation confounds	Domain 5: evaluation design and reliability
Constitutional AI and RLAIF	Domain 1: Claude-specific alignment approach
The four priorities and three principals	Domain 4: system prompt authority; Domain 2: tool and retrieval trust
The escalation ladder: prompt → retrieve → tools → fine-tune	Domain 1 and Domain 4: the single most frequent scenario shape
Fine-tuning risks: staleness, upgrades, alignment erosion	Domain 1: architecture decisions and total cost of ownership

CERTIFICATION NOTE

Exam tips carried forward

- **Fine-tuning is almost always the wrong answer.** When it appears as an option, look for prompting, retrieval or tool use instead. It is correct only when the scenario explicitly rules those out.
- When a scenario needs **current or proprietary knowledge**, the answer is retrieval, never fine-tuning.
- When a scenario needs **exactness** — arithmetic, lookups, writes — the answer is tool use.
- Expect at least one question where a **metric improves while quality degrades**. The answer involves the evaluation set, not the model.
- Options claiming the model "learns from the conversation" or "remembers previous sessions" are wrong. Parameters are frozen; memory is architecture.
- Remember the priority order: **safe, ethical, compliant with guidelines, helpful**. Helpfulness is fourth.
- You are the **operator**. System prompt is operator authority; user turns, tool results and retrieved documents are not.

7.19 Practice Questions

1. A base model asked "What is the capital of France?" may reply with more questions because: (a) It is broken. (b) It continues documents rather than answering. (c) It lacks the knowledge. (d) Its context is full.

2. Which stage consumes the overwhelming majority of total training compute? (a) SFT. (b) RLHF. (c) Pre-training. (d) Evaluation.

3. Supervised fine-tuning uses which objective? (a) A new ranking loss. (b) The same next-token loss, on different data. (c) Policy gradient. (d) Contrastive loss.

4. The main limitation of SFT that motivates RLHF is that demonstrations: (a) Are too numerous. (b) Cannot express that one response is better than another. (c) Cost nothing. (d) Require reinforcement learning.

5. In RLHF, human labellers primarily produce: (a) Ideal responses. (b) Comparisons between responses. (c) Reward values. (d) Gradients.

6. A reward model outputs: (a) A token sequence. (b) A single scalar score. (c) A probability distribution over the vocabulary. (d) An embedding.

7. Under Bradley–Terry, adding a constant to every reward: (a) Changes all rankings. (b) Changes nothing, because only differences matter. (c) Doubles the loss. (d) Breaks the sigmoid.

8. The KL penalty in RLHF exists to: (a) Speed up training. (b) Stop the policy drifting into regions where the reward model is unreliable. (c) Reduce token cost. (d) Enforce the constitution.

9. A reward model scores 100% on its training distribution and 56% where length and quality disagree. This indicates: (a) A good model. (b) Reward hacking driven by a confound in the data. (c) Insufficient epochs. (d) A learning-rate problem.

10. The most reliable fix for that reward model is: (a) More epochs. (b) Decorrelating the preference data. (c) A length penalty at selection time. (d) A bigger feature space.

11. Best-of-n selection using a hacked reward model will: (a) Correct the hack. (b) Systematically select the most hacked candidate. (c) Have no effect. (d) Reduce cost.

12. RLAIF differs from RLHF in that: (a) It skips the reward model. (b) A model rather than a human produces the preference labels. (c) It needs no data. (d) It removes the KL term.

13. Constitutional AI's supervised phase involves the model: (a) Ranking responses. (b) Critiquing and revising its own outputs against written principles. (c) Generating pre-training data. (d) Tokenizing input.

14. Claude's constitution was published under which licence? (a) MIT. (b) Creative Commons CC0. (c) Apache 2.0. (d) It is not public.

15. The constitution's stated priority order is: (a) Helpful, safe, ethical, compliant. (b) Broadly safe, broadly ethical, compliant with guidelines, genuinely helpful. (c) Compliant, helpful, safe, ethical. (d) Ethical, helpful, safe, compliant.

16. In the constitution's terms, a company building on the Claude API is: (a) A user. (b) An operator. (c) A principal equal to Anthropic. (d) A labeller.

17. An instruction inside a retrieved document should be treated as: (a) Operator authority. (b) Data, not instruction. (c) Equal to the system prompt. (d) A tool call.

18. A team needs the assistant to know last week's pricing. The correct approach is: (a) Fine-tune weekly. (b) Retrieval. (c) A longer system prompt. (d) Raise `max_tokens`.

19. Which is *not* a documented risk of fine-tuning? (a) Alignment erosion. (b) Must be redone at model upgrades. (c) Requires curated data. (d) Permanently increases context window size.

20. "The model has learned from our conversations over the past month" is: (a) True for aligned models. (b) False — parameters are frozen after training; memory is architecture you build. (c) True with prompt caching. (d) True above 100,000 tokens.

7.20 Answers and Explanations

1 — (b). The base model completes documents; in a quiz-sheet context more questions are the probable continuation. (a) misreads correct behaviour as malfunction. (c) is wrong — the knowledge is present, the behaviour is not. (d) is unrelated to the phenomenon.

2 — (c). Pre-training dominates compute and data; stages 2 and 3 are comparatively tiny. (a) and (b) are the stages that shape behaviour, not the ones that consume the compute. (d) is not a training stage.

3 — (b). Nothing changes mechanically; only the data distribution does. That is precisely why SFT is cheap. (a), (c) and (d) all describe different objectives belonging to other stages or methods.

4 — (b). A demonstration asserts one correct answer and cannot encode comparative judgement. (a) is backwards — demonstrations are scarce. (c) is false; they are expensive expert work. (d) states a consequence, not the limitation.

5 — (b). The shift from writing answers to comparing them is the core of RLHF, because comparison is faster and more consistent. (a) describes SFT. (c) is the reward model's output, not the human's. (d) is computed, not labelled.

6 — (b). One scalar per prompt-response pair. (a) describes the policy. (c) describes a language model's output layer. (d) describes an encoder, as in Chapter 5.

7 — (b). `sigmoid(r(a) − r(b))` is invariant to an additive constant. This is why reward scores are ordinal only. (a) is the opposite. (c) and (d) are inventions.

8 — (b). Without it the policy finds adversarial inputs to its own grader. (a) is false; it constrains rather than accelerates. (c) is unrelated. (d) confuses a mathematical regulariser with a values document.

9 — (b). The gap between in-distribution and hard-set accuracy is the definition of the phenomenon. (a) ignores the second number. (c) and (d) are tuning explanations for what is a data problem — the exact error section 7.14 warns about.

10 — (b). Fix 1 took the length weight from +0.654 to −0.006 and hard-set accuracy from 56.5% to 100%. (a) would deepen the fit to the confound. (c) is Fix 2, which worked at exactly one coefficient. (d) does not address the confound.

11 — (b). Selection amplifies whatever the scorer believes, so it picks the most hacked candidate from each batch — potentially worse than sampling once. (a) inverts this. (c) ignores that selection has an effect. (d) is false; it multiplies cost.

12 — (b). The labeller changes; the pipeline of Figure 7.2 does not. (a) is false — a reward model is still trained. (c) is false. (d) is false.

13 — (b). Self-critique and revision produce the SFT data; ranking belongs to the reinforcement phase. (a) is that later phase. (c) and (d) are unrelated stages.

14 — (b). Published under Creative Commons CC0 1.0, placing it in the public domain. (a) and (c) are software licences not used here. (d) contradicts the publication.

15 — (b). Broadly safe, broadly ethical, compliant with Anthropic's guidelines, genuinely helpful. (a), (c) and (d) all shuffle the order; the examinable point is that helpfulness comes last.

16 — (b). The constitution names Anthropic, operators and users; if you build on the API you are the operator. (a) describes your end users. (c) overstates it. (d) is not a principal.

17 — (b). Retrieved content carries no operator authority; treating it as instruction is the indirect prompt injection surface. (a) and (c) are the vulnerability. (d) confuses content with a control-flow block.

18 — (b). Knowledge that changes belongs in retrieval, where updating a document is instant. (a) is expensive and immediately stale. (c) does not scale and still requires manual updating. (d) is unrelated.

19 — (d). Fine-tuning does not change the context window, which is an architectural property of the model. (a), (b) and (c) are all genuine documented risks discussed in section 7.8.

20 — (b). Parameters are frozen at the end of stage 3; every apparent memory is context you supplied or storage you built. (a) confuses alignment with learning. (c) confuses a latency optimisation with weight updates. (d) invents a threshold.

7.21 Ten Practical Exercises

1. Write out, for a system you work on, which behaviours come from pre-training, which from alignment, and which from your prompt.
2. Take a refusal you have encountered and classify it against the four priorities.
3. Draft a system prompt that explicitly counteracts verbosity, sycophancy and over-hedging, and measure output token counts before and after.
4. Build a ten-item evaluation set where a plausible confound in your domain is deliberately broken.
5. Identify a metric in your organisation that could improve while quality degrades, and say how.
6. For a fine-tuning proposal you have seen, work down the escalation ladder and document what each rung would cost.
7. Audit your codebase for anywhere user input reaches the system prompt.
8. Write the safety evaluation you would run before and after any fine-tune.
9. Read the four-priority list and predict three behaviours your product will encounter as a result.
10. Estimate the annual maintenance cost of a fine-tune, including one model upgrade.

7.22 Five Challenge Labs

1. **A second confound.** Add formality as a confounded variable alongside length, and determine whether the reward model can be made to separate three correlated factors. Report how much decorrelated data it takes.
2. **LLM-as-judge, gamed.** Build a simple model-based grader for a task of your choosing, then deliberately craft an input that scores highly and is bad. Document the exploit.
3. **A KL penalty you can see.** Implement a toy policy over a small discrete output space, optimise it against a deliberately flawed reward, and plot output quality against β. Identify the window where quality peaks.
4. **Preference data audit.** Take a real dataset of paired judgements from your organisation and measure the correlation between preference and length, formatting and formality.
5. **Constitution-guided evaluation.** Using the published constitution, write ten test prompts that probe the boundary between priorities 3 and 4, and record actual behaviour.

7.23 Interview Questions

1. Walk me through how a raw pre-trained model becomes something you can ship.
2. Why is supervised fine-tuning not enough on its own?
3. What is a reward model, and why are its absolute scores meaningless?
4. What does the KL penalty do, and what happens without it?
5. What is reward hacking? Give an example you could detect in your own metrics.
6. How would you tell whether a reward model or evaluation metric is confounded?
7. What is Constitutional AI, and how does it differ from standard RLHF?
8. A stakeholder wants to fine-tune. What do you ask before agreeing?
9. Why is retrieval preferred to fine-tuning for knowledge?

10. What does it mean that you are an "operator", and what follows from it architecturally?

7.24 Summary

A deployable language model is made in three stages, and confusing them is the source of a great deal of expensive architectural error. Pre-training is self-supervised next-token prediction over an enormous corpus; it consumes almost all of the compute and produces almost all of the capability, but it produces a document-continuer rather than an assistant. Supervised fine-tuning follows, using the identical loss on a small curated set of demonstrations, and converts that continuer into something that answers questions — cheaply, because the capability was already there and only needed to be pointed at the right behaviour.

Supervised fine-tuning alone cannot express comparative judgement. A demonstration says *this is good*; it cannot say *this is better than that*, and it gives no gradient at all for what to avoid. Preference optimisation closes the gap by changing what humans produce: comparisons instead of answers. Those comparisons train a reward model under the Bradley–Terry formulation, in which only reward *differences* are meaningful, and the reward model then guides policy optimisation — restrained by a KL penalty which is not an implementation detail but the component that stops the policy from finding adversarial inputs against its own grader.

The central practical lesson of this chapter is that the reward model is a learned, imperfect proxy, and we measured exactly how imperfect. Trained on preference data where better answers happened also to be longer — the confound that arises naturally in real human labelling — our reward model reached 100% accuracy on its training distribution and 56.5% where length and quality disagreed, which is barely above chance. Used for best-of-n selection it ranked a 42-word answer carrying two facts above a 30-word answer carrying four. Repairing the data fixed it completely, driving the length weight from +0.654 to −0.006 and flipping filler words from rewarded to penalised. Patching the symptom with a length penalty worked at exactly one coefficient value and failed on both sides of it — a fix you can only apply once you already know the answer. Reward hacking is a data problem wearing an algorithm problem's clothes.

RLAIF replaces the human labeller with a model guided by written principles, and Anthropic's implementation is Constitutional AI, used in Claude's training since 2023. The constitution published on 22 January 2026 under a CC0 dedication marks a shift from a list of rules to an explanation of reasons, on the argument that a model needs to understand *why* in order to generalise to situations no rule anticipated — with a small set of absolute hard constraints retained. Anthropic states that Claude itself uses the document to generate synthetic training data including rankings of candidate responses, and is candid that a gap remains between the document's intentions and actual behaviour.

Two details from that document belong in your architectural vocabulary. The four priorities — broadly safe, broadly ethical, compliant with Anthropic's guidelines, genuinely helpful — are ordered, and helpfulness comes last, which explains refusals that no amount of prompt engineering will move. And the three principals — Anthropic, operators, users — tell you that as an operator your authority lives in the system prompt and does not extend to text arriving in user turns, tool results or retrieved documents. That distinction is the foundation of everything Chapter 58 says about prompt injection. Finally, and most usefully day to day: work down the escalation ladder. Prompt, then few-shot, then retrieve, then use tools, then cache and downshift the model — and only then, rarely, and with a named owner for the

retraining pipeline, fine-tune.

KEY TAKEAWAYS

1. **Capability comes from pre-training; behaviour comes from alignment.** Two different stages, two different cost profiles.
2. **A base model continues documents.** Everything assistant-like was added in stages 2 and 3.
3. **SFT reuses the next-token loss on curated data** — nothing new mechanically, which is why it is cheap and effective.
4. **RLHF exists because comparisons express what demonstrations cannot.**
5. **Reward models are ordinal.** Only differences matter; absolute scores are meaningless across runs.
6. **The KL penalty stops the policy attacking its own grader.** It is load-bearing, not decorative.
7. **Reward hacking is a data problem.** Our model: 100% in-distribution, 56.5% where the confound broke.
8. **Best-of-n amplifies the scorer's beliefs**, including its pathologies.
9. **The constitution's priorities are ordered and helpfulness is fourth** — which explains refusals prompt engineering cannot move.
10. **You are the operator.** System prompt carries your authority; retrieved content and user turns do not.

7.25 Glossary

Term	Definition
Alignment	Training a model to behave helpfully, honestly and harmlessly, beyond raw capability.
Base model	The output of pre-training: broad knowledge, no instruction-following behaviour.
Best-of-n	Sampling several responses and selecting the highest-scoring under a reward model.
Bradley–Terry	The model `P(a preferred) = sigmoid(r(a) − r(b))`, used to train reward models from pairs.
Constitutional AI	Anthropic's alignment approach using a written constitution for self-critique and AI-generated preferences.
Hard constraint	A behaviour the constitution forbids absolutely, rather than weighing against other values.
Instruction tuning	Supervised fine-tuning on instruction-and-response demonstrations.
KL penalty	A term penalising divergence of the policy from the SFT model, controlled by `β`.
Operator	In the constitution's terms, a party building on the API. You.
Policy	The language model being optimised during reinforcement learning.
Pre-training	Self-supervised next-token training on a very large corpus.
Preference pair	A prompt with a chosen and a rejected response; the unit of RLHF data.
Principal	A party whose interests the model weighs: Anthropic, operators, users.

Term	Definition
Reward hacking	Achieving high measured reward through behaviour that defeats the reward's intent.
Reward model	A network scoring a prompt-response pair with one scalar, trained on preference pairs.
RLAIF	Reinforcement learning from AI feedback: a model supplies the preference labels.
RLHF	Reinforcement learning from human feedback.
Supervised fine-tuning	Continued training on curated `(prompt, response)` demonstrations.
Sycophancy	The tendency to agree with the user, a common artefact of preference training.

7.26 Further Reading

Primary and canonical sources.

- Bai et al. (2022), *Constitutional AI: Harmlessness from AI Feedback* (Anthropic). The primary source for section 7.7.
- Anthropic, *Claude's new constitution* (22 January 2026) and the full constitution at `anthropic.com/constitution`, released under CC0. The authoritative source for the priorities and principals of sections 7.7.1 and 7.7.2.
- Christiano et al. (2017), *Deep Reinforcement Learning from Human Preferences*. The origin of the reward-model-from-comparisons approach.
- Ouyang et al. (2022), *Training Language Models to Follow Instructions with Human Feedback*. The canonical description of the three-stage pipeline.
- Bradley and Terry (1952), *Rank Analysis of Incomplete Block Designs*. The statistical model underneath every reward model.
- Rafailov et al. (2023), *Direct Preference Optimization*. The main alternative to the reward-model-plus-policy-gradient pipeline.
- Anthropic system cards, which document where measured behaviour departs from the constitution's intentions.

Where to go next in this book.

Chapter 8 closes Part 1 by taking up what alignment does not solve: evaluation, hallucination, safety and responsible AI — including why a model optimised for plausibility confabulates, and how you measure that rather than hope about it. From there, Part 2 turns from how models are built to how they behave: Chapter 9 on the anatomy of an LLM, Chapter 10 on capabilities and hard limits, and Chapter 13 on reasoning models, where the best-of-n trade-off of section 7.15 becomes a first-class architectural choice. The threads opened here run through Chapter 21 on system prompts as operator authority, Chapter 57 on evaluation harnesses built to catch the confounds of section 7.6, and Chapter 58 on prompt injection, which is the principal hierarchy under attack.